Week 6 – Vulnerability Findings Register

DG Interns Hub – Cybersecurity Internship

Web Security & Vulnerability Testing (VAPT Basics)

Target	Local DVWA application
Target URL	`http://127.0.0.1/dvwa/`
Tools	Burp Suite Community Edition; OWASP ZAP
Environment	Local Kali Linux lab
Register status	Preliminary – evidence-based review required

Scope note

This educational assessment is limited to an intentionally vulnerable local DVWA environment. Demonstrations in the lab are distinguished from unvalidated scanner alerts and topics not confirmed as practically tested.

Contents

Section and finding references

1 Document Information

2 Purpose

3 Findings Summary

4 Detailed Findings

4.1 WEB-01 – SQL Injection

4.2 WEB-02 – Reflected Cross-Site Scripting (XSS)

4.3 WEB-03 – Stored Cross-Site Scripting (XSS)

4.4 WEB-04 – OWASP ZAP Scanner Alerts

4.5 WEB-05 – Broken Authentication

4.6 WEB-06 – Security Misconfiguration

5 Recommended Remediation and Retesting Plan

6 Overall Assessment

7 References

Evidence and validation status are retained as documented in the source register. No remediation is represented as completed unless supported by retesting evidence.

1. Document Information

Field	Details
Internship	DG Interns Hub – Cybersecurity Internship
Week	Week 6
Project	Web Security & Vulnerability Testing (VAPT Basics)
Target	Local DVWA application
Target URL	`http://127.0.0.1/dvwa/`
Testing Tools	Burp Suite Community Edition, OWASP ZAP
Testing Environment	Local Kali Linux lab
Assessment Type	Educational lab assessment
Register Status	Preliminary – evidence-based review required

2. Purpose

This register documents the observations from the Week 6 web application security exercises. It records the testing evidence, potential security impact, recommended remediation, and validation status.

The results are limited to the intentionally vulnerable local DVWA environment. They must not be interpreted as evidence that a production application has the same vulnerabilities.

3. Findings Summary

ID	Finding	Evidence Status	Priority
WEB-01	SQL Injection	Demonstrated in DVWA	High attention
WEB-02	Reflected Cross-Site Scripting	Demonstrated in DVWA	High attention
WEB-03	Stored Cross-Site Scripting	Demonstrated in DVWA	High attention
WEB-04	OWASP ZAP Scanner Alerts	Alerts reported; individual validation pending	Pending triage
WEB-05	Broken Authentication	Theory reviewed; dedicated practical test not confirmed	Not assessed
WEB-06	Security Misconfiguration	Potential observations require individual validation	Pending triage

> ***Priority note:*** *The attention labels above are not formal severity ratings. A real severity rating requires assessment of the specific weakness, affected functionality, likelihood, impact, and supporting evidence.*

4. Detailed Findings

WEB-01 – SQL Injection

Category: Injection **Affected Component:** DVWA SQL Injection module **Environment:** Local lab, DVWA security set to Low

Description

SQL Injection occurs when an application incorporates untrusted input into a database query without safely separating data from query instructions.

During the lab exercise, a test payload caused DVWA to return multiple records rather than only the expected individual result.

Evidence

- The SQL Injection module was tested in the local DVWA application.
- A normal input returned a record.
- The test payload `1' OR '1'='1' #` returned multiple records.
- A corresponding HTTP request was also captured and replayed using Burp Suite Repeater.

Evidence location:

- `02-Screenshots/01-SQL-Injection/`
- `02-Screenshots/04-Burp-Suite/`

Potential Impact

In a real vulnerable application, SQL Injection could expose or alter database information and potentially bypass application-level data restrictions. The actual impact depends on the query, database privileges, and application design.

Recommended Remediation

1. Use parameterized queries or prepared statements.
2. Avoid constructing SQL queries through string concatenation.
3. Apply least-privilege permissions to the application's database account.
4. Handle database errors without exposing sensitive technical details.
5. Retest the affected functionality after remediation.

Validation Status

Demonstrated in the intentionally vulnerable lab. No production vulnerability or successful remediation is claimed.

WEB-02 – Reflected Cross-Site Scripting (XSS)

Category: Injection – Cross-Site Scripting **Affected Component:** DVWA Reflected XSS module **Environment:** Local lab, DVWA security set to Low

Description

Reflected XSS can occur when an application returns untrusted input in a response without appropriate context-aware output encoding.

During testing, a script payload triggered a browser alert in the DVWA Reflected XSS module.

Evidence

- A test script was entered into the Reflected XSS input.

- The browser alert demonstrated script execution in the local test application.
- A screenshot was captured as evidence.

Evidence location:

- `02-Screenshots/02-Reflected-XSS/`

Potential Impact

In a real application, XSS could allow malicious script execution in a user's browser within the context of the affected site. Impact depends on the vulnerable page, available privileges, browser controls, and application behavior.

Recommended Remediation

1. Apply context-aware output encoding.
2. Validate input according to the application's expected data format.
3. Use safe templating and DOM APIs.
4. Avoid unsafe insertion of untrusted data into HTML.
5. Use Content Security Policy as an additional defense layer.
6. Retest the affected rendering context after remediation.

Validation Status

Demonstrated in the intentionally vulnerable lab. No production vulnerability or successful remediation is claimed.

WEB-03 – Stored Cross-Site Scripting (XSS)

Category: Injection – Cross-Site Scripting **Affected Component:** DVWA guestbook / Stored XSS module **Environment:** Local lab, DVWA security set to Low

Description

Stored XSS can occur when an application stores untrusted input and later displays it without appropriate output encoding or sanitization.

During the exercise, a script payload submitted through the DVWA guestbook triggered a browser alert when the stored content was rendered.

Evidence

- A script payload was submitted to the guestbook.
- The stored content was rendered by the application.
- A browser alert demonstrated script execution.
- A screenshot was captured.

Evidence location:

- `02-Screenshots/03-Stored-XSS/`

Potential Impact

Stored XSS may affect users who view the affected content. Depending on the application, it may enable unauthorized actions within a user's browser context or expose accessible page data.

Recommended Remediation

1. Apply context-aware output encoding whenever stored content is rendered.
2. Use a maintained allow-list sanitizer if user-authored HTML is intentionally supported.
3. Prefer safe rendering APIs over direct HTML insertion.

4. Review existing stored content where appropriate.
5. Retest every relevant page that displays the stored data.

Validation Status

Demonstrated in the intentionally vulnerable lab. No production vulnerability or successful remediation is claimed.

WEB-04 – OWASP ZAP Scanner Alerts

Category: Automated web security scan observations **Affected Component:** Local DVWA application **Tool:** OWASP ZAP 2.17.0

Description

OWASP ZAP was used to scan the local DVWA application. The exported HTML report summarized 13 alerts.

Reported Alert Summary

Alert Category	Count
High	0
Medium	5
Low	5
Informational	3
Total	**13**

The report summary indicated 5 high-confidence alerts and 8 medium-confidence alerts.

Evidence

- An OWASP ZAP scan was initiated against the local application.
- The scan was stopped manually before completion.
- An HTML report was exported.
- Screenshots of the alert summary and report generation were saved.

Evidence location:

- `02-Screenshots/05-OWASP-ZAP/`
- `03-Reports/2026-10-13-ZAP-Report-127.0.0.1.html`

Interpretation

The counts represent the alerts reported by the partial scan. They do not establish that 13 exploitable vulnerabilities exist.

The scan was not completed, and individual alerts require manual review. The absence of High alerts does not establish that the application is secure.

Recommended Actions

1. Review each alert's description, affected URL, evidence, and confidence.
2. Manually validate relevant alerts within the authorized lab.
3. Identify false positives and informational observations.
4. Record validated findings separately from unconfirmed scanner alerts.
5. Complete an appropriate scan when practical.
6. Export and retain the final report if a complete scan is performed.

Validation Status

Partial scan completed; alert validation pending.

WEB-05 – Broken Authentication

Category: Identification and Authentication Failures **Assessment Status:** Not assessed through a dedicated practical test

Description

Broken authentication can occur when login, identity verification, session management, or authentication controls are incorrectly implemented.

The topic was covered in the OWASP notes, but the available project evidence does not confirm that a dedicated authentication assessment was performed.

Potential Impact

Depending on the weakness, an application may be exposed to account compromise, session misuse, or unauthorized access.

Recommended Security Controls

1. Enforce secure authentication and appropriate password policies.
2. Use secure session management and regenerate session identifiers after authentication.
3. Set session cookies with appropriate `HttpOnly`, `Secure`, and `SameSite` attributes.
4. Implement appropriate rate limiting and monitoring for repeated login failures.
5. Apply multi-factor authentication where appropriate.
6. Test authentication and session behavior within an authorized lab.

Validation Status

Theory documented; practical testing not confirmed. No vulnerability is claimed as found.

WEB-06 – Security Misconfiguration

Category: Security Misconfiguration **Assessment Status:** Requires evidence review

Description

Security misconfiguration can result from insecure defaults, unnecessary services or methods, excessive information disclosure, missing security controls, or incorrectly configured permissions.

OWASP ZAP was used to scan the local DVWA application. However, the available alert summary does not identify which individual alerts have been manually confirmed as security misconfigurations.

Potential Impact

Depending on the specific configuration, an application may disclose unnecessary information, expose unintended functionality, or provide weaker security protections.

Recommended Actions

1. Review individual ZAP alerts and their supporting evidence.
2. Confirm the affected URL, response, and actual configuration behavior.
3. Disable unnecessary functionality where appropriate.
4. Configure security headers based on application requirements.
5. Review server defaults, permissions, error handling, and exposed information.
6. Retest after any configuration changes.

Validation Status

Potential observations require individual validation. No specific misconfiguration is claimed as confirmed based solely on the alert summary.

5. Recommended Remediation and Retesting Plan

Finding	Proposed Action	Retesting Method	Current Status
WEB-01 – SQL Injection	Implement parameterized database queries	Repeat authorized test cases and verify inputs are treated as data	Proposed
WEB-02 – Reflected XSS	Implement context-aware output encoding	Retest the affected output context using safe test cases	Proposed
WEB-03 – Stored XSS	Encode stored content when rendered; sanitize supported HTML where needed	Retest submission and all relevant display locations	Proposed
WEB-04 – ZAP Alerts	Review and validate individual alerts; complete scanning as appropriate	Manual reproduction and follow-up scan	Pending
WEB-05 – Broken Authentication	Review authentication and session controls	Perform an authorized authentication test plan	Not tested
WEB-06 – Security Misconfiguration	Validate alerts and review server/application configuration	Verify configuration and retest affected behavior	Pending

These actions are recommendations. Do not mark a finding as fixed until a remediation has actually been implemented and supported by retesting evidence.

6. Overall Assessment

The Week 6 exercises demonstrated SQL Injection, Reflected XSS, and Stored XSS behavior in the intentionally vulnerable local DVWA application. Burp Suite was used to inspect and replay an HTTP request, and OWASP ZAP produced an HTML report from a partial scan.

The practical work illustrates the value of combining manual testing, proxy-based request inspection, and automated scanning. However, scanner alerts require triage, and a partial scan cannot establish that an application is secure.

The next steps are to validate relevant ZAP alerts, document any additional tests actually completed, and ensure the final report and presentation accurately reflect the available evidence.

7. References

- OWASP Top 10
- OWASP Web Security Testing Guide
- OWASP SQL Injection Prevention Cheat Sheet
- OWASP Cross Site Scripting Prevention Cheat Sheet
- OWASP ZAP Documentation
- PortSwigger Web Security Academy

Document: Vulnerability Findings Register **Project:** DG Interns Hub – Cybersecurity Internship **Week:** 6 – Web Security & Vulnerability Testing (VAPT Basics) **Purpose:** Evidence-based findings, remediation planning, and security learning